



Interclasarea tablourilor | Tehnica Two Pointers

Operații cu mulțimi · Sliding Window

Andrei Mihai Ciobanu

Algolymp Summer School

Interclasarea tablourilor

Operații cu mulțimi

Tehnica Two Pointers

Sliding Window

Recapitulare

Resurse

Probleme de antrenament

Interclasarea tablourilor

Problema

Avem doi vectori **sortați**: $a[1 \dots n]$ și $b[1 \dots m]$.

Interclasare

Construiește $c[1 \dots n+m]$, sortat, cu toate elementele din a și b .

Abordare	Complexitate
Concatenează + sortează	$O((n + m) \log(n + m))$
Interclasare clasică	$O(n + m)$

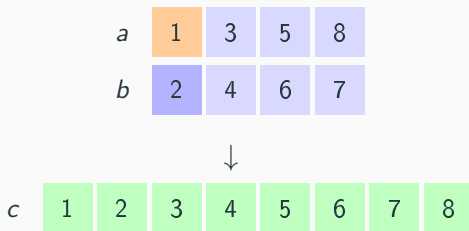
Interclasarea exploatează că a și b sunt **deja sortate** — este baza algoritmului **Merge Sort**.

Algoritmul

Doi indici i (în a) și j (în b). La fiecare pas alegem minimum:

- $a[i] \leq b[j]$: punem $a[i]$ în c , avansăm i
- $a[i] > b[j]$: punem $b[j]$ în c , avansăm j
- Când un vector se termină, copiem tot restul celuilalt

Exemplu: $a = [1, 3, 5, 8]$, $b = [2, 4, 6, 7]$:



Fiecare element e procesat exact o dată $\Rightarrow O(n + m)$

Implementare

```
1 // atata timp cat avem elemente in ambele siruri,
2 // alegem cel mai mic dintre cele ramase
3 int i = 1, j = 1, k = 0;
4 while (i <= n && j <= m) {
5     if (a[i] <= b[j]) {
6         c[++k] = a[i++];
7     } else {
8         c[++k] = b[j++];
9     }
10 }
11
12 // la final, vor ramane elemente doar in a[]...
13 while (i <= n) {
14     c[++k] = a[i++];
15 }
16
17 // ...sau doar in b[]
18 while (j <= m) {
19     c[++k] = b[j++];
20 }
```

Operații cu mulțimi

Operații cu mulțimi pe vectori sortați

Dacă a și b sunt sortați și **fără duplicate**, putem calcula operații de mulțimi în $O(n + m)$:

Operație	Notatie	Conținut rezultat
Reuniune	$A \cup B$	elemente din a sau b (fără duplicate)
Intersecție	$A \cap B$	elemente comune
Diferență	$A \setminus B$	cele din a , fără cele care apar în b

Același principiu ca interclasarea: doi indici care avansează simultan.

Reuniune ($A \cup B$)

```
1 // cat timp mai avem elemente in ambele siruri
2 int i = 1, j = 1, k = 0;
3 while (i <= n && j <= m) {
4     if (a[i] < b[j]) {
5         c[++k] = a[i++];
6     } else if (b[j] < a[i]) {
7         c[++k] = b[j++];
8     } else { // a[i] == b[j]
9         c[++k] = a[i++];
10        j++;
11    }
12 }
13
14 // copiază ce a rămas în a[]
15 while (i <= n) {
16     c[++k] = a[i++];
17 }
18
19 // copiază ce a rămas în b[]
20 while (j <= m) {
21     c[++k] = b[j++];
22 }
```

Elementele comune apar o singură dată în c .

Intersecție ($A \cap B$)

```
1 // cat timp mai avem elemente in ambele siruri
2 int i = 1, j = 1, k = 0;
3 while (i <= n && j <= m) {
4     if (a[i] == b[j]) {
5         c[++k] = a[i++];
6         j++;
7     } else if (a[i] < b[j]) {
8         i++;
9     } else {
10        j++;
11    }
12 }
13
14 // c[1..k] contine elementele comune
```

Adăugăm în *c* doar elementele care apar în **ambii** vectori.

Diferență ($A \setminus B$)

```
1 // cat timp mai avem elemente in ambele siruri
2 int i = 1, j = 1, k = 0;
3 while (i <= n && j <= m) {
4     if (a[i] < b[j]) {
5         c[++k] = a[i++];
6     } else if (a[i] == b[j]) {
7         i++; j++;
8     } else {
9         j++;
10    }
11 }
12
13 // copiaza elementele ramase din a[] care nu apar in b[]
14 while (i <= n) {
15     c[++k] = a[i++];
16 }
```

Adăugăm elementele din a care **nu apar** în b .

Tehnica Two Pointers

Two Pointers — ideea generală

Mentăinem doi indici l și r pe un vector. Niciun indice nu dă înapoi.

De ce $O(n)$?

Fiecare element e adăugat o singură dată (când r trece prin el) și eliminat o singură dată (când l trece prin el) $\Rightarrow$ cel mult $2n$ pași $\Rightarrow O(n)$.

Există două variante în funcție de cum se mișcă pointerii:

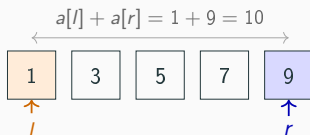
Variantă	Mișcare		Condiție
Pointeri din capete	$l \rightarrow$	$\leftarrow r$	vector sortat
Pointeri în același sens	$l \rightarrow$	$r \rightarrow$	elemente pozitive

Pointeri din capete — exemplu

$a = [1, 3, 5, 7, 9]$ (sortat), $K = 12$. Pornim cu $l = 1$, $r = 5$.

Întrebare

Dacă $a[l] + a[r] \leq K$ și vectorul e sortat, ce putem spune despre perechile $(l, l+1)$, $(l, l+2)$, $\dots$, $(l, r-1)$?



l	r	$a[l] + a[r]$	Acțiune	cnt
1	5	$1+9 = 10 \leq 12$	$cnt += r - l = 4$, $l++$	4
2	5	$3+9 = 12 \leq 12$	$cnt += r - l = 3$, $l++$	7
3	5	$5+9 = 14 > 12$	$r--$	7
3	4	$5+7 = 12 \leq 12$	$cnt += r - l = 1$, $l++$	8

$l \geq r$ — se oprește. Răspuns: $cnt = 8$.

Numărul de perechi cu suma $\leq K$

Sortăm a . Menținem $l = 1$ (capăt stâng) și $r = n$ (capăt drept).

Cazul $a[l] + a[r] \leq K$ — toate perechile $(l, l+1), \dots, (l, r)$ sunt valide

Vectorul e sortat: $a[j] \leq a[r]$ pentru orice $j \leq r$, deci

$a[l] + a[j] \leq a[l] + a[r] \leq K$. Adăugăm $r - l$ perechi, avansăm l .

Cazul $a[l] + a[r] > K$ — retragem r

$a[l]$ e cel mai mic element din stânga; dacă suma lui cu $a[r]$ depășește K , nicio pereche cu capăt drept r nu e validă.

Ne oprim când $l \geq r$.

Implementare — Numărul de perechi cu suma $\leq K$

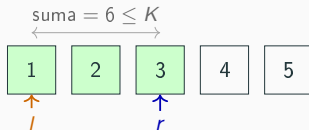
```
1  sort(a + 1, a + n + 1); // sortam vectorul
2
3  int l = 1, r = n;
4  long long cnt = 0;
5  while (l < r) {
6      if (a[l] + a[r] <= K) {
7          cnt += r - l;    // perechile (l, l+1), ..., (l, r)
8          l++;
9      } else {
10         r--;
11     }
12 }
```

Pointeri în același sens — exemplu

$a = [1, 2, 3, 4, 5]$, $K = 9$ — câte secvențe $[l, r]$ au suma ≤ 9 ?

Întrebare

Dacă $[l, r]$ are suma $\leq K$ (elemente pozitive!), ce putem spune despre $[l, l]$, $[l, l+1]$, $\dots$, $[l, r-1]$?



l	r	suma	Secvențe noi	ans
1	3	6	$[1, 1]$, $[1, 2]$, $[1, 3]$	3
2	4	9	$[2, 2]$, $[2, 3]$, $[2, 4]$	6
3	4	7	$[3, 3]$, $[3, 4]$	8
4	5	9	$[4, 4]$, $[4, 5]$	10
5	5	5	$[5, 5]$	11

Numărul de secvențe cu suma $\leq K$

Problemă: câte secvențe $[l, r]$ au $a[l] + \dots + a[r] \leq K$? (elemente pozitive)

Extindem r cât putem — fereastra $[l, r]$ e mereu validă (suma $\leq K$). Secvențele $[l, l]$, $[l, l+1]$, $\dots$, $[l, r]$ sunt valide.

Adăugăm $r - l + 1$ la contor, apoi eliminăm $a[l]$ și avansăm l .

Implementare — Numărul de secvențe cu suma $\leq K$

```
1 long long answer = 0, sum = 0;
2 for (int l = 1, r = 0; l <= n; ++l) {
3     // extindem r cat putem
4     while (r + 1 <= n && sum + a[r + 1] <= K) {
5         sum += a[++r];
6     }
7     // toate secventele [l, l], [l, l + 1],
8     // ..., [l, r] sunt valide
9     answer += r - l + 1;
10    // eliminam l din secventa
11    sum -= a[l];
12 }
```

Numărul de secvențe cu suma $\geq K$

Problemă: câte secvențe $[l, r]$ au $a[l] + \dots + a[r] \geq K$? (elemente pozitive)

Extindem r cât e nevoie — căutăm cel mai mic r pentru care suma $\geq K$. Secvențele $[l, r]$, $[l, r+1]$, $\dots$, $[l, n]$ sunt valide.

Adăugăm $n - r + 1$ la contor, apoi eliminăm $a[l]$ și avansăm l .

Observație: se poate calcula și ca $\frac{n(n+1)}{2} - cnt_{<K}$, unde $cnt_{<K}$ = nr. secvențe cu suma $< K$.

Implementare — Numărul de secvențe cu suma $\geq K$

```
1 long long answer = 0, sum = 0;
2 for (int l = 1, r = 0; l <= n; ++l) {
3     // extindem r cat timp e nevoie
4     while (r + 1 <= n && sum < K) {
5         sum += a[++r];
6     }
7     // toate secventele [l, r], [l, r + 1],
8     // ..., [l, n] sunt valide
9     if (sum >= K) {
10        answer += n - r + 1;
11    }
12    // eliminam l din secventa
13    sum -= a[l];
14 }
```

Sliding Window

Sliding Window

Sliding Window: fereastra are mereu exact k elemente consecutive.

Exemplu

Suma maximă a k elemente consecutive din $a[1 \dots n]$.

- Calculăm suma primei ferestre $[1, k]$
- La fiecare pas: adăugăm $a[r]$, scoatem $a[r - k]$
- Actualizăm maximul — fără a recalcula de la zero

Fiecare element e adăugat și scos o singură dată $\Rightarrow O(n)$.

Implementare — Sliding Window

```
1  const int nmax = 100000;
2  int a[nmax + 1];
3  int n, k;
4
5  long long suma = 0, maxSuma = 0;
6  for (int i = 1; i <= k; i++) {
7      suma += a[i];
8  }
9  maxSuma = suma;
10
11 for (int r = k + 1; r <= n; r++) {
12     suma += a[r] - a[r - k]; // adaugam dreapta, scoatem
    stanga
13     if (suma > maxSuma) {
14         maxSuma = suma;
15     }
16 }
```

Recapitulare

Recapitulare

Tehnică	Complexitate	Condiție
Interclasare	$O(n + m)$	ambii sortați
Operații cu mulțimi	$O(n + m)$	ambii sortați
Pointeri din capete	$O(n)$	vector sortat
Pointeri în același sens	$O(n)$	elemente pozitive
Sliding Window	$O(n)$	—

- Cheia: nici un indice nu dă înapoi

Resurse

- **pbinfo.ro** — Interclasarea tablourilor (RO)
- **AlgoGenius** — Tehnica Two Pointers (RO)
- **USACO Guide** — Two Pointers (EN)
- **Infobits Academy F1** — p. 79 (RO)
- **Competitive Programmer's Handbook** — cap. 8 (EN)

Probleme de antrenament

Interclasare

- **Interclasare** (pbinfo #241)
- **Interclasare1** (pbinfo #250)
- **Multimi1** (pbinfo #530)

Probleme de antrenament — Two Pointers

1. **Pair Sum Window Count** (AlgoCode #4277)
2. **Sum of Two Values** (CSES #1640)
3. **palindrom** (AlgoCode #1747 · OJI 2023 VII)
4. **secvmin** (AlgoCode #503 · ONI 2023 VII)
5. **Mapat** (AlgoCode #630 · ONI 2026 VII)
6. **criptografie** (AlgoCode #2373 · ONI 2019 VIII)
7. **kmajo** (AlgoCode #3072 · ONI 2024 VIII)